Guía de Actividades Práctico-Experimentales Nro. 003

1. Datos Generales

Integrantes	- Arelys Ajila - Josue Asanza - Kevin Nole - Cristhian Valderde
Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Sincronización de Procesos: Exclusión Mutua, Semáforos y Problemas Clásicos de Concurrencia
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de de 2026 09 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Implementar una barrera de sincronización con variables de condición que detenga a los hilos hasta que todos alcancen el punto de encuentro.

- Implementar mecanismos de exclusión mutua utilizando mutex y cerrojos (locks) para proteger secciones críticas en programas multihilo, garantizando que no se produzcan condiciones de carrera al acceder a recursos compartidos.
- Aplicar semáforos como herramienta de sincronización para controlar el acceso a recursos limitados y coordinar la ejecución de hilos según restricciones específicas de orden y disponibilidad.
- Resolver el problema clásico del Productor-Consumidor utilizando buffers compartidos y mecanismos de sincronización que aseguren que el productor no escriba en un buffer lleno y el

consumidor no lea de un buffer vacío.

- Implementar la solución al problema de Lectores-Escritores, distinguiendo entre las variantes con prioridad para lectores y prioridad para escritores, analizando las implicaciones de cada enfoque en términos de inanición y concurrencia.

3. Materiales y reactivos:

Guia practica

Lista de

ejercicios

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

Paso 1: Planeación de los sistemas.

En esta sección se definieron los recursos y dependencias que se usaron para los diferentes sistemas. En primer lugar, se definió a Java como lenguaje de programación y Spring Boot como marco de trabajo.

Paso 2: Instalación de dependencias

Aquí se procedió a instalar las dependencias con la ayuda de docker en algunos casos.

Ejercicio 1. Control de acceso a un estacionamiento — Semáforo contador

Objetivo: Aplicar semáforos contadores para controlar el acceso concurrente a un recurso limitado.

Situación: Un estacionamiento dispone de 4 espacios. Llegan 10 vehículos de manera concurrente. Cada vehículo debe esperar si no existe un espacio disponible. Cuando un vehículo sale, libera el espacio para otro vehículo.

Procedimiento

1. Identifique el recurso compartido.
2. Determine la cantidad máxima de vehículos que pueden permanecer dentro del estacionamiento.
3. Cree un semáforo contador inicializado en 4.
4. Cree un proceso vehículo(id).
5. El vehículo deberá solicitar un espacio antes de ingresar.
6. Si no existen espacios disponibles, deberá permanecer bloqueado.
7. Una vez que obtenga un espacio, deberá simular su permanencia.
8. Al salir, deberá liberar el espacio mediante signal().
9. Cree 10 procesos vehículo.
10. Analice qué sucede cuando llegan más de 4 vehículos simultáneamente.

Pregunta de análisis:

¿Qué ocurriría si el semáforo se inicializa en 1? ¿Y si se inicializará en 10?

Ejercicio 2. Cajero automático compartido — Exclusión mutua

Objetivo: Implementar exclusión mutua para evitar condiciones de carrera sobre un recurso compartido.

Situación: Una cuenta bancaria tiene un saldo inicial de $1000. Tres procesos realizan operaciones simultáneamente sobre la misma cuenta:

- Proceso 1: retiro de $300.
- Proceso 2: retiro de $250.
- Proceso 3: depósito de $200.

El saldo debe modificarse de forma segura.

Procedimiento

1. Identifique la variable compartida.
2. Determine qué operaciones pueden ejecutarse simultáneamente.
3. Cree un mutex para proteger el saldo.
4. Antes de consultar o modificar el saldo, el proceso debe adquirir el mutex.
5. Realice la operación correspondiente.
6. Libere el mutex después de actualizar el saldo.
7. Cree los tres procesos.
8. Ejecute el programa varias veces.
9. Analice qué podría ocurrir si se elimina el mutex.
10. Compare el comportamiento con y sin exclusión mutua

Ejercicio 3. Fábrica de ensamblaje — Productor-Consumidor

Objetivo: Implementar el patrón productor-consumidor utilizando una cola limitada y semáforos.

Situación:Una fábrica tiene una línea de producción con capacidad para almacenar 5 piezas temporalmente.

- Los productores fabrican piezas.
- Los consumidores toman piezas para ensamblarlas.
- Un productor no puede colocar una pieza si el almacén está lleno.
- Un consumidor no puede retirar una pieza si el almacén está vacío.

Procedimiento

1. Cree una cola con capacidad máxima de 5 elementos.
2. Cree un semáforo espacios inicializado en 5.
3. Cree un semáforo elementos inicializado en 0.
4. Cree un mutex para proteger la cola.
5. Implemente el proceso productor.
6. El productor deberá esperar un espacio disponible.
7. Debe adquirir el mutex antes de modificar la cola.
8. Debe insertar una pieza.
9. Debe liberar el mutex.
10. Debe indicar que existe una nueva pieza.
11. Implemente el proceso consumidor.
12. El consumidor deberá esperar hasta que exista una pieza.
13. Debe adquirir el mutex antes de retirar una pieza.
14. Debe retirar la pieza.
15. Debe liberar el mutex.
16. Debe indicar que existe un espacio disponible.

Pregunta de análisis:

¿Qué sucede si espacios se inicializan en 0? ¿Qué sucede si elementos se inicializan en 5?

Ejercicio 4. Cruce de vehículos — Control de acceso en una dirección

Objetivo: Coordinar procesos que comparten un recurso y evitar que dos grupos incompatibles accedan simultáneamente.

Situación: Un puente permite circular en ambos sentidos, pero solamente puede existir una dirección de circulación activa a la vez.

Pueden existir varios vehículos cruzando hacia el norte, pero no puede haber vehículos hacia el norte y hacia el sur simultáneamente.

Procedimiento

1. Identifique el recurso compartido.
2. Defina una variable que indique la dirección actual.
3. Mantenga un contador de vehículos que están cruzando.
4. Proteja las variables mediante un mutex.
5. Un vehículo deberá verificar la dirección antes de entrar.
6. Si existen vehículos en dirección contraria, deberá esperar.
7. Si puede entrar, incrementará el contador correspondiente.
8. Después de cruzar, disminuirá el contador.
9. Cuando no queden vehículos, el puente podrá cambiar de dirección.
10. Analice el problema de inanición.

Reto: modificar el algoritmo para evitar que una dirección monopolice el puente.

Ejercicio 5. Centro de impresión — Sincronización de procesos

Objetivo: Utilizar semáforos para establecer un orden determinado de ejecución entre procesos.

Situación:

Un centro de impresión tiene tres procesos:

- Diseño
- Revisión
- Impresión

Para cada documento debe cumplirse obligatoriamente:

DISEÑO → REVISIÓN → IMPRESIÓN

No se puede revisar un documento que todavía no ha sido diseñado y tampoco se puede imprimir antes de que haya sido revisado.

Procedimiento

1. Identifique los procesos participantes.
2. Cree un semáforo para cada etapa.
3. Permita inicialmente que el proceso de diseño pueda ejecutarse.
4. Diseño deberá habilitar a revisión.
5. La revisión deberá habilitar la impresión.
6. Impresión deberá habilitar nuevamente a diseño.
7. Repita el ciclo para 3 documentos.
8. Verifique que el orden siempre sea correcto.
9. Analice qué ocurre si se elimina uno de los signal().

6. Resultados esperados:

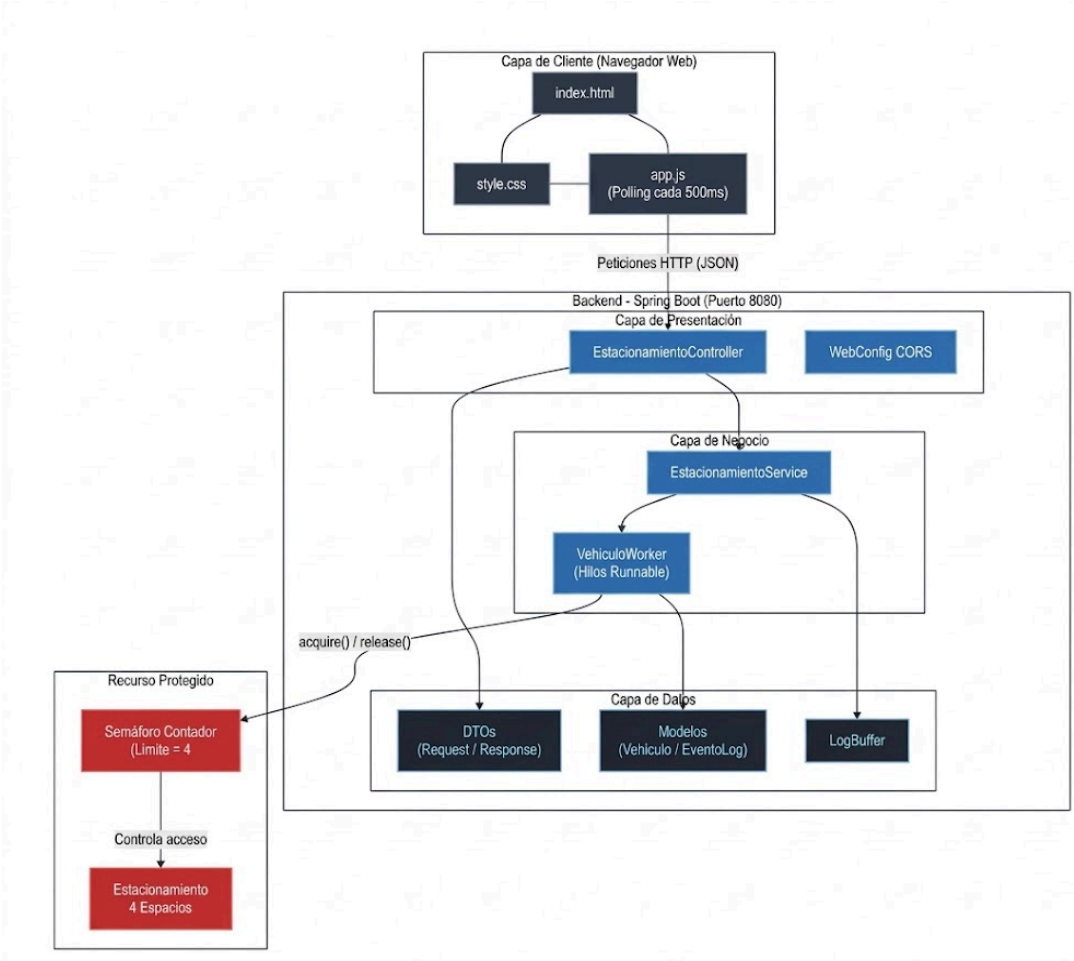
Ejercicio 1. Control de acceso a un estacionamiento — Semáforo contador
Repositorio:
https://github.com/Cristhval/APE3_SistemasDistribuidos/tree/master/Estacionamiento

1. Datos Generales y Tecnologías Utilizadas

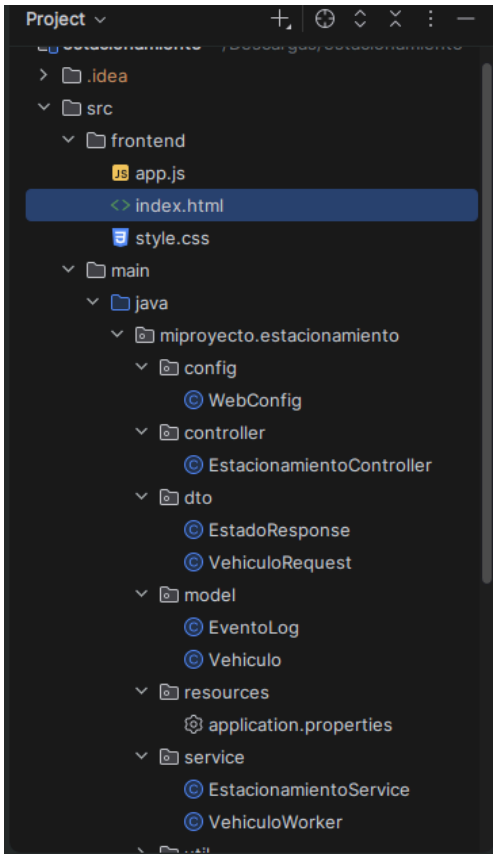
Arquitectura: Cliente-Servidor

- Backend: Java 17, Spring Boot
- Frontend: HTML5, CSS3 y JavaScript nativo
- Entorno de Ejecución: Ejecución local nativa en Kubuntu, Backend en puerto 8081 y Frontend en puerto 8080.

2. Diagrama de Arquitectura

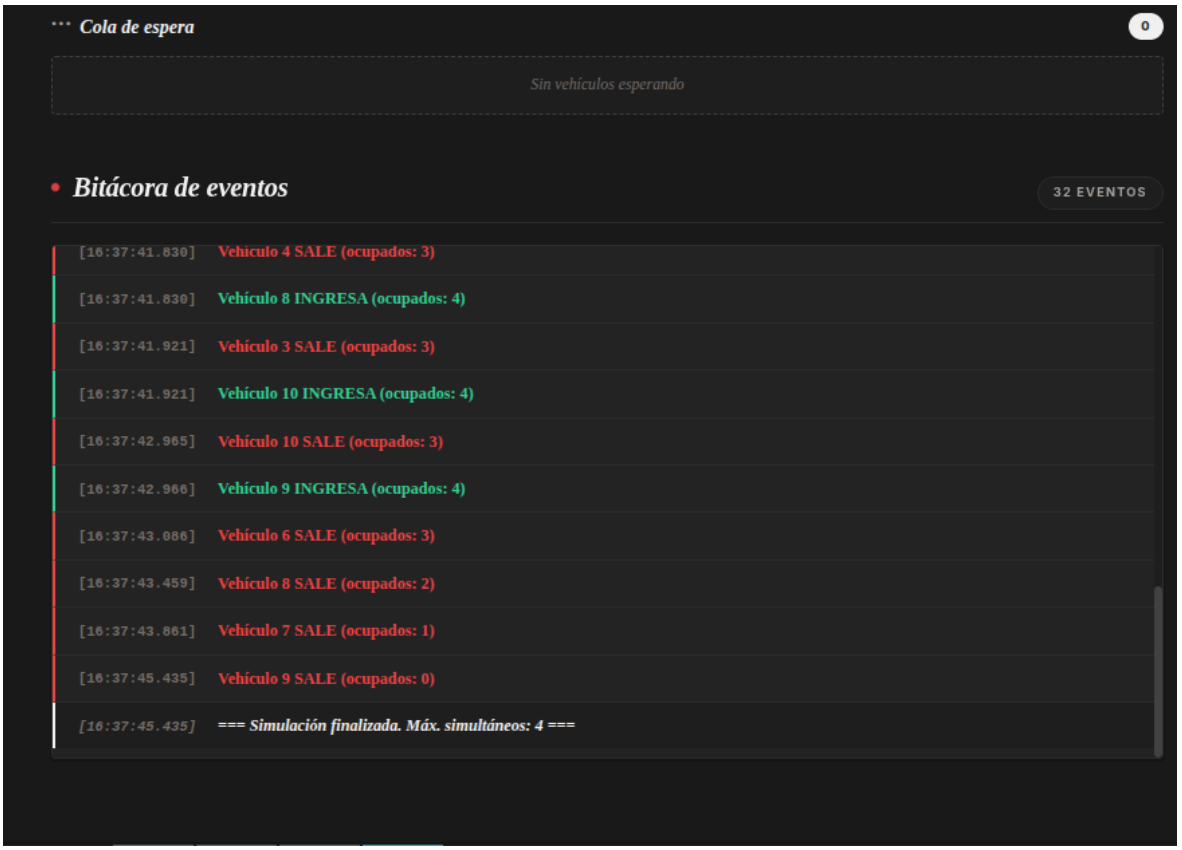


3. Organización y Estructura de Archivos


Project
.idea
src
frontend
app.js
index.html
style.css
main
java
miproyecto.estacionamiento
config
WebConfig
controller
EstacionamientoController
dto
EstadoResponse
VehiculoRequest
model
EventoLog
Vehiculo
resources
application.properties
service
EstacionamientoService
VehiculoWorker

3. Interfaz


ESTACIONAMIENTO CONCURRENTE
Semáforo contador · Exclusión mutua · Simulación en tiempo real
Ejecutando...
CONFIGURACIÓN DEL EXPERIMENTO
VEHÍCULOS
10
CAPACIDAD
4
INICIAR SIMULACIÓN
LIMPIAR
OCUPADOS
4
Vehículos dentro
CAPACIDAD
4
Espacios totales
MÁXIMO
4
Simultáneos históricos
TIEMPO
2s
Transcurrido
Vista del estacionamiento
30%



Pregunta de análisis:

¿Qué ocurriría si el semáforo se inicializara en 1? ¿Y si se inicializara en 10?

Si el semáforo se inicializa en 1 este funcionara como un semáforo binario por lo que solo un vehículo podrá ingresar a la vez por lo que solo se permitirá que ingrese solamente un auto, pero si se iniciara en 10 todos los vehículos podrian ingresar a la vez pero como la capacidad es solo de 4 habria sobrepoblacion siendo que solo se dispone de 4 lugares.

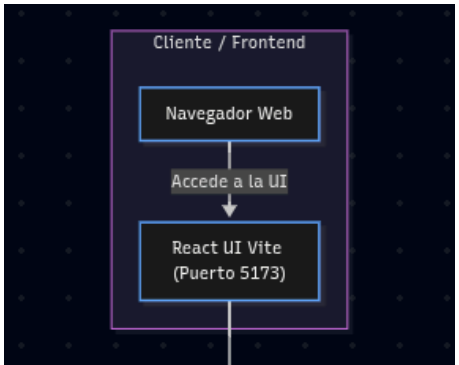
Ejercicio 2: Cajero Automático Compartido — Exclusión Mutua

Repositorio: https://github.com/Josue082004/Sistemas-Distribuidos.git

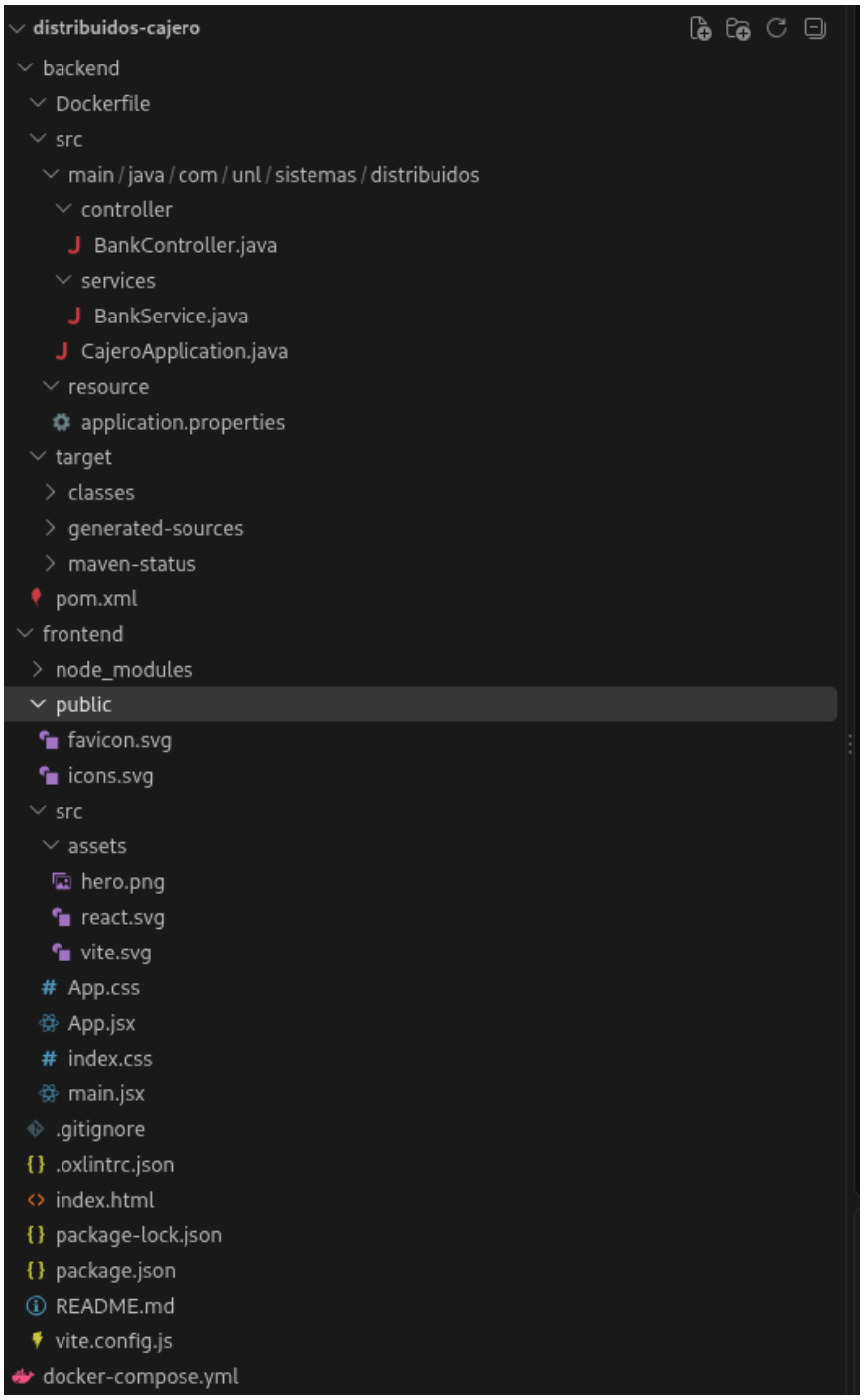
1. Datos Generales y Tecnologías Utilizadas

- Arquitectura: Cliente-Servidor
- Backend: Java 17, Spring Boot
- Frontend: React (Vite).
- Entorno de Ejecución: Ejecución local nativa en Debian, Backend en puerto 8080 y Frontend en puerto 5173.

2. Diagrama de Arquitectura



3. Organización y Estructura de Archivos



4. Metodología Aplicada

La metodología de desarrollo fue incremental:

Se implementó el servicio BankService aplicando el patrón Singleton gestionado por Spring , se utilizó BigDecimal para evitar problemas de precisión en los cálculos monetarios.

backend > src > main > java > com > unl > sistemas > distribuidos > services > BankService.java

```java
package com.unl.sistemas.distribuidos.service;

import org.springframework.stereotype.Service;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

@Service
public class BankService {
    private BigDecimal saldo = new BigDecimal("1000.00").setScale(2, RoundingMode.HALF_UP);
    private final Lock mutexSaldo = new ReentrantLock();

    public boolean retirar(int id, BigDecimal cantidad) {
        mutexSaldo.lock();
        try {
            if (cantidad == null || cantidad.compareTo(BigDecimal.ZERO) <= 0 || cantidad.scale() > 2) {
                return false;
            }

            cantidad = cantidad.setScale(2, RoundingMode.HALF_UP);
            System.out.println("Proceso " + id + " intenta retirar \$" + cantidad);

            if (saldo.compareTo(cantidad) >= 0) {
                saldo = saldo.subtract(cantidad).setScale(2, RoundingMode.HALF_UP);
                System.out.println("-> Retiro exitoso. Saldo actual: \$" + saldo);
                return true;
            } else {
                System.out.println("-> Saldo insuficiente.");
                return false;
            }
        } finally {
            mutexSaldo.unlock();
        }
    }

    public boolean depositar(int id, BigDecimal cantidad) {
        mutexSaldo.lock();
        try {
            if (cantidad == null || cantidad.compareTo(BigDecimal.ZERO) <= 0 || cantidad.scale() > 2) {
                return false;
            }
            cantidad = cantidad.setScale(2, RoundingMode.HALF_UP);
            System.out.println("Proceso " + id + " deposita \$" + cantidad);

            saldo = saldo.add(cantidad).setScale(2, RoundingMode.HALF_UP);
            System.out.println("-> Depósito exitoso. Saldo actual: \$" + saldo);
            return true;
            mutexSaldo.unlock();
        }
    }

    public BigDecimal getSaldo() {
        mutexSaldo.lock();
        try {
            return saldo.setScale(2, RoundingMode.HALF_UP);
        } finally {
            mutexSaldo.unlock();
        }
    }
}
```

Se diseñó BankController para exponer endpoints seguros con códigos de estado HTTP adecuados.

backend > src > main > java > com > unl > sistemas > distribuidos > controller > BankController.java

```java
package com.unl.sistemas.distribuidos.controller;

import com.unl.sistemas.distribuidos.service.BankService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.math.BigDecimal;

@RestController
@RequestMapping("/api1/cajero")
@CrossOrigin(origins = "http://localhost:5173")
public class BankController {

    private final BankService bankService;

    public BankController(BankService bankService) {
        this.bankService = bankService;
    }

    @GetMapping("/saldo")
    public ResponseEntity<BigDecimal> obtenerSaldo() {
        return ResponseEntity.ok(bankService.getSaldo());
    }

    @PostMapping("/operar")
    public ResponseEntity<String> realizarOperacion(
            @RequestParam int id,
            @RequestParam BigDecimal cantidad,
            @RequestParam boolean esDeposito) {

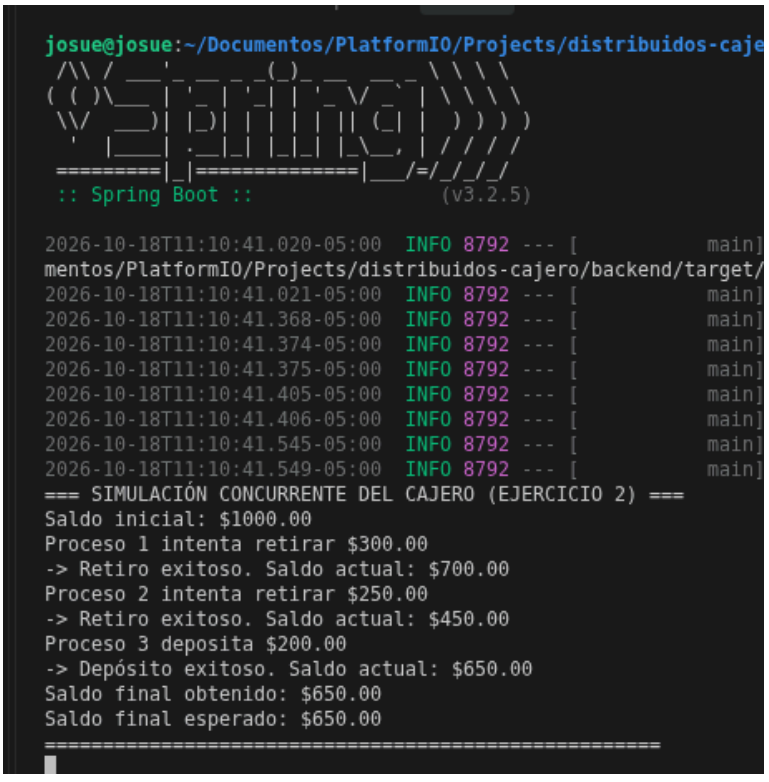
        if (cantidad == null || cantidad.compareTo(BigDecimal.ZERO) <= 0 || cantidad.scale() > 2) {
            return ResponseEntity.badRequest().body("Error: Cantidad invalida, debe ser mayor a 0 y con máximo 2 decimales");
        }

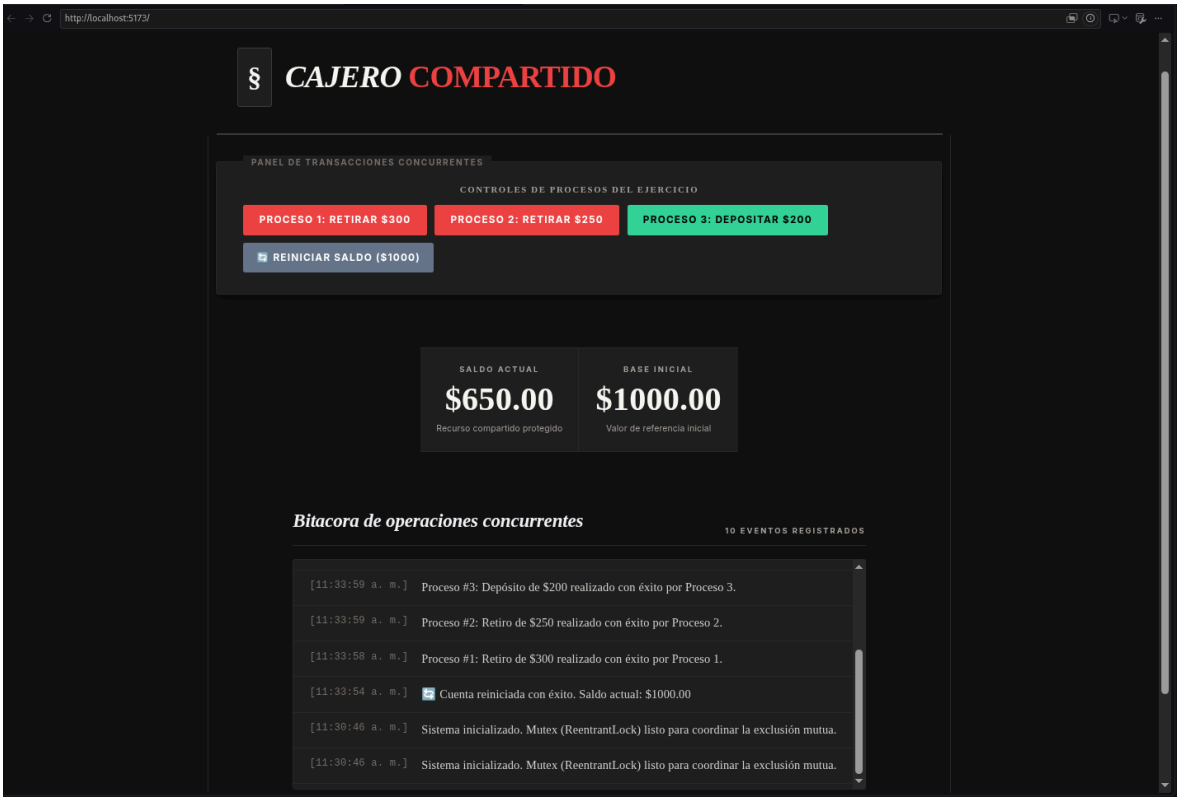
        if (esDeposito) {
            boolean exito = bankService.depositar(id, cantidad);
            if (exito) {
                return ResponseEntity.ok("Deposito de $" + cantidad + " realizado con exito.");
            } else {
                return ResponseEntity.badRequest().body("Error al procesar el deposito.");
            }
        } else {
            boolean exito = bankService.retirar(id, cantidad);
            if (exito) {
                return ResponseEntity.ok("Retiro de $" + cantidad + " realizado con exito.");
            } else {
                return ResponseEntity.status(422).body("Error: Saldo insuficiente en la cuenta para este retiro.");
            }
        }
    }
}
```

Se construyó una interfaz moderna en React integrando un panel de control y una bitácora de eventos.



Ejecución






```
=== SIMULACIÓN CONCURRENTE DEL CAJERO (EJERCICIO 2) ===
Saldo inicial: $1000.00
Proceso 1 intenta retirar $300.00
-> Retiro exitoso. Saldo actual: $700.00
Proceso 2 intenta retirar $250.00
-> Retiro exitoso. Saldo actual: $450.00
Proceso 3 deposita $200.00
-> Depósito exitoso. Saldo actual: $650.00
Saldo final obtenido: $650.00
Saldo final esperado: $650.00
=====================================================
2026-10-18T11:28:51.254-05:00  INFO 10376 --- [nio-8080
2026-10-18T11:28:51.254-05:00  INFO 10376 --- [nio-8080
2026-10-18T11:28:51.255-05:00  INFO 10376 --- [nio-8080
-> Saldo reiniciado a su valor inicial: $1000.00
Proceso 1 intenta retirar $300.00
-> Retiro exitoso. Saldo actual: $700.00
Proceso 2 intenta retirar $250.00
-> Retiro exitoso. Saldo actual: $450.00
Proceso 3 deposita $200.00
-> Depósito exitoso. Saldo actual: $650.00
-> Saldo reiniciado a su valor inicial: $1000.00
Proceso 1 intenta retirar $300.00
-> Retiro exitoso. Saldo actual: $700.00
Proceso 2 intenta retirar $250.00
-> Retiro exitoso. Saldo actual: $450.00
Proceso 3 deposita $200.00
-> Depósito exitoso. Saldo actual: $650.00
-> Saldo reiniciado a su valor inicial: $1000.00
Proceso 1 intenta retirar $300.00
-> Retiro exitoso. Saldo actual: $700.00
Proceso 2 intenta retirar $250.00
-> Retiro exitoso. Saldo actual: $450.00
Proceso 3 deposita $200.00
-> Depósito exitoso. Saldo actual: $650.00
```

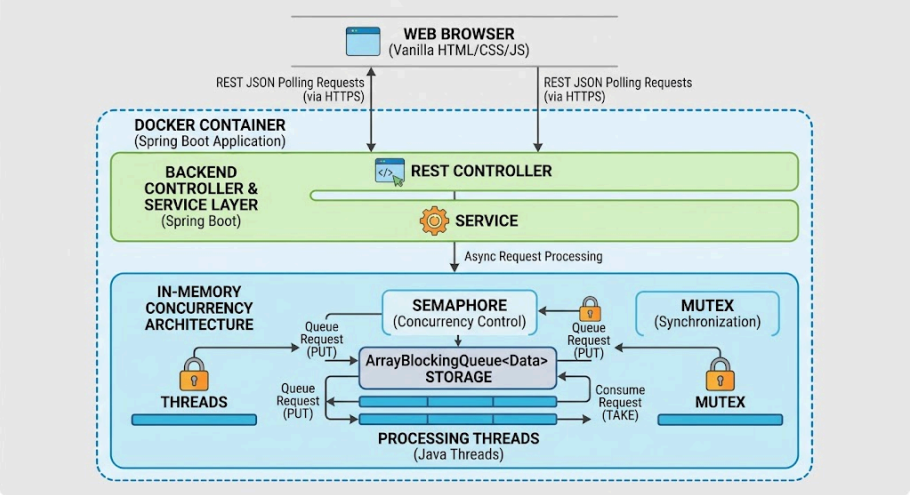
Pregunta de análisis:

¿Por qué el acceso a saldo debe protegerse con un mutex?

Para evitar condiciones de carrera, si múltiples hilos o procesos intentan leer y modificar la variable saldo de manera simultánea sin exclusión mutua, las operaciones se superponen en la memoria, provocando pérdidas de actualización, saldos inconsistentes y resultados no deterministas.

Ejercicio 3: Fábrica, modelo Producto-Consumidor

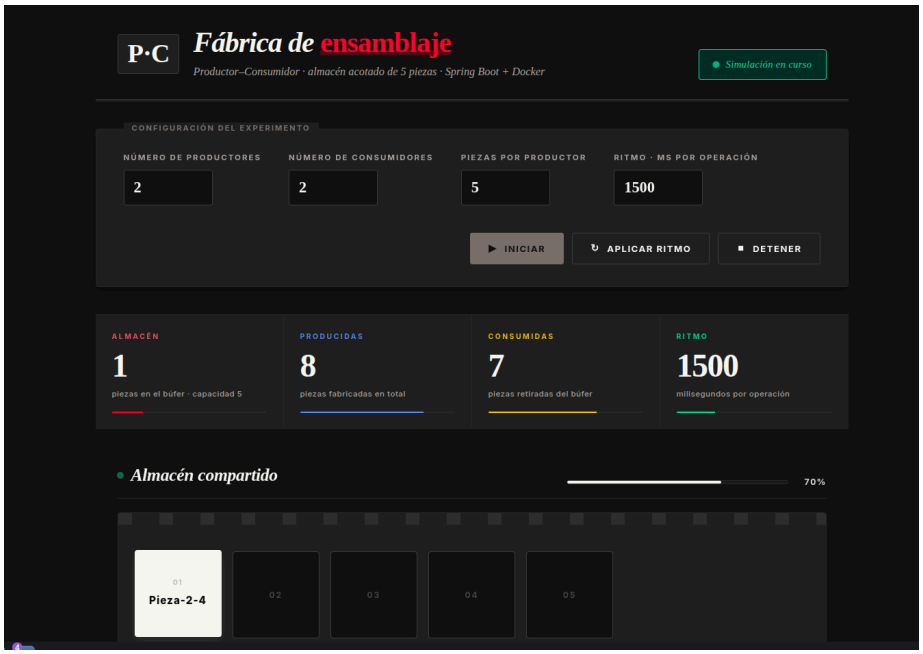
1. **Repositorio:** https://github.com/Cristhval/APE3_SistemasDistribuidos/tree/master/Producto-Consumidor/Practica%203
2. **Datos y tecnologías utilizadas**
   - **Arquitectura:** Cliente-servidor, implementada como una aplicación monolítica.
   - **Backend:** Java 17 y Spring Boot 3.2.5.
   - **Frontend:** HTML5, CSS3 y JavaScript nativo (Vanilla JavaScript).
   - **Entorno de Ejecución:** Linux Mint 22.3 como sistema anfitrión y Docker Compose para ejecutar el backend dentro de un contenedor Alpine Linux con OpenJDK 17. Backend y frontend servidos desde http://localhost:8080
3. **Arquitectura**



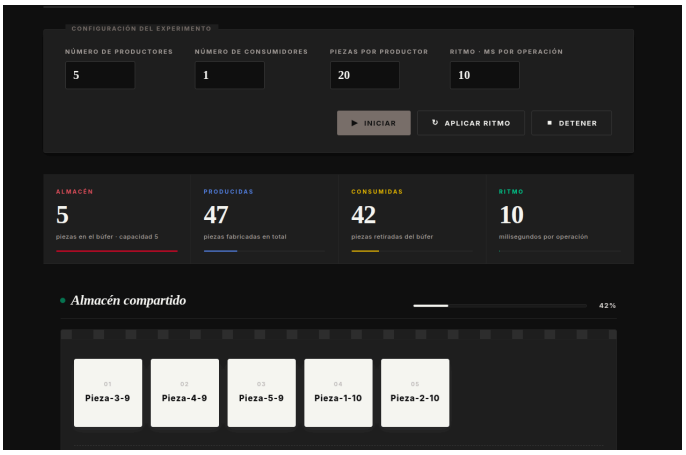
4. **Interfaz**

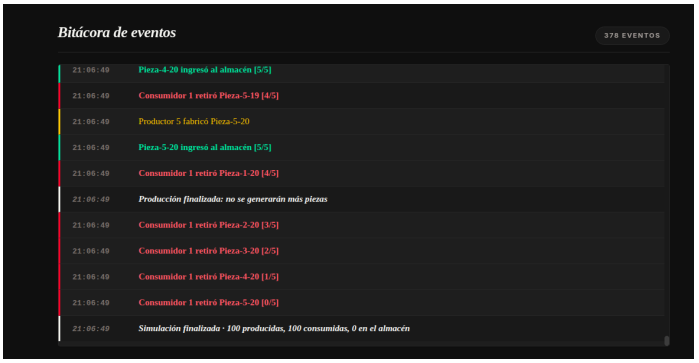


5. **Funcionamiento**
   - **2 Productores, 2 consumidores, 5 piezas por productor cada 1500 ms.**



   - **5 Productores, 1 consumidor, 20 piezas por productor cada 10 ms.**





6. **Pregunta de análisis:**
   - **¿Qué sucede si espacios se inicializan en 0? ¿Qué sucede si elementos se inicializan en 5?**

     En el primer caso, el semáforo indicaría que no hay espacios disponibles en el almacén. Por ello, los productores deben esperar hasta que un consumidor retire una pieza para liberar espacio, sin embargo, debido a que el almacén comienza vacío, los consumidores no tienen piezas por retirar y el flujo no puede ejecutarse correctamente.

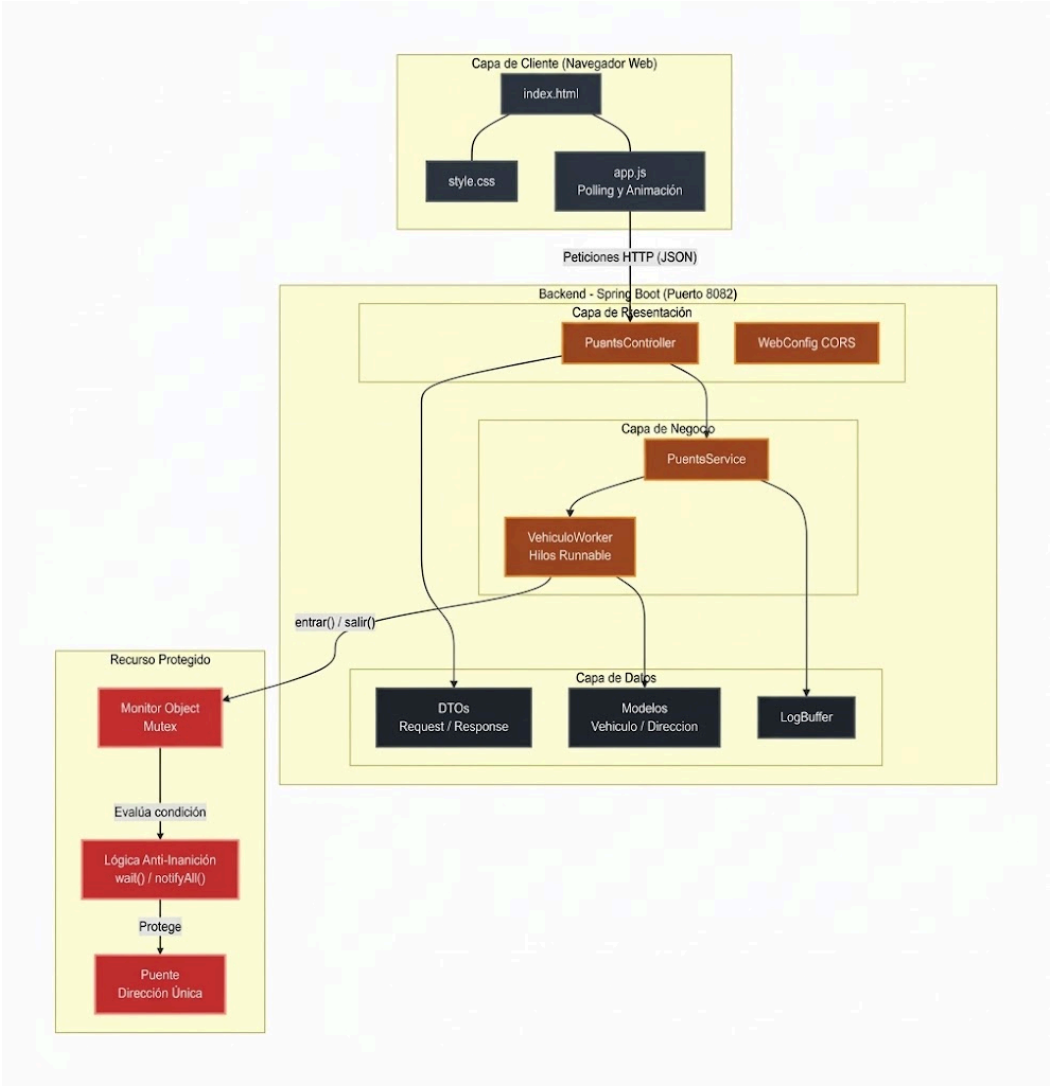
     Por otra parte, en caso que los elementos comienzan en 5, el semáforo indicaría que existen 5 elementos disponibles para consumir, aunque el almacén comienza vacío. Si los consumidores intentan retirar piezas deben comprobar que existen en el almacén primero, por lo que la sincronización inicial no representa el estado real del almacén.

Ejercicio 4. Cruce de vehículos — Control de acceso en una dirección
Repositorio:
https://github.com/Cristhval/APE3_SistemasDistribuidos/tree/master/Puente

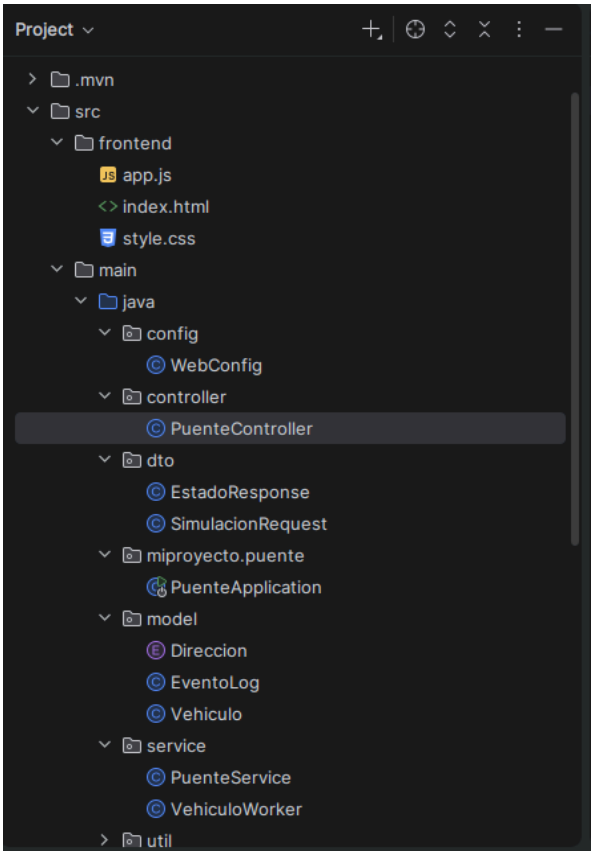
1. Datos Generales y Tecnologías Utilizadas
Arquitectura: Cliente-Servidor

- Backend: Java 17, Spring Boot
- Frontend: HTML5, CSS3 y JavaScript nativo
- Entorno de Ejecución: Ejecución local nativa en Kubuntu, Backend en puerto 8082 y Frontend en puerto 8082.

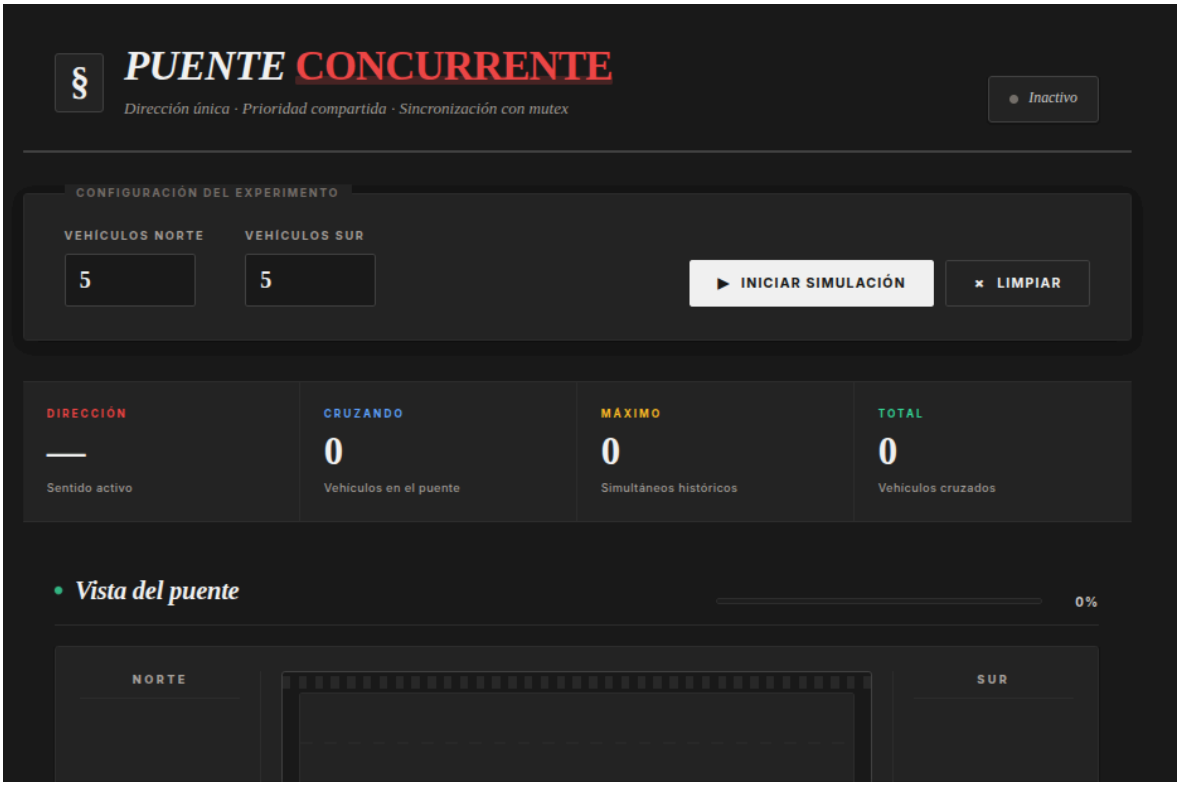
2. Diagrama de Arquitectura

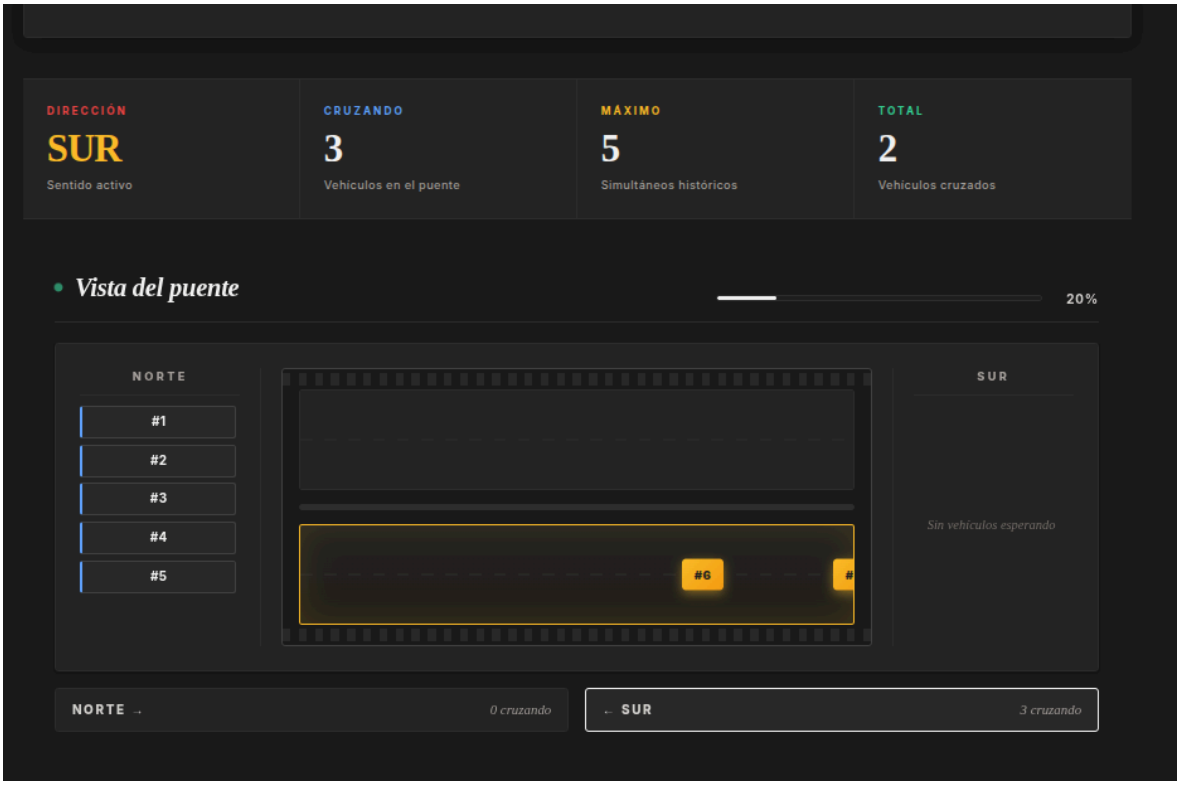


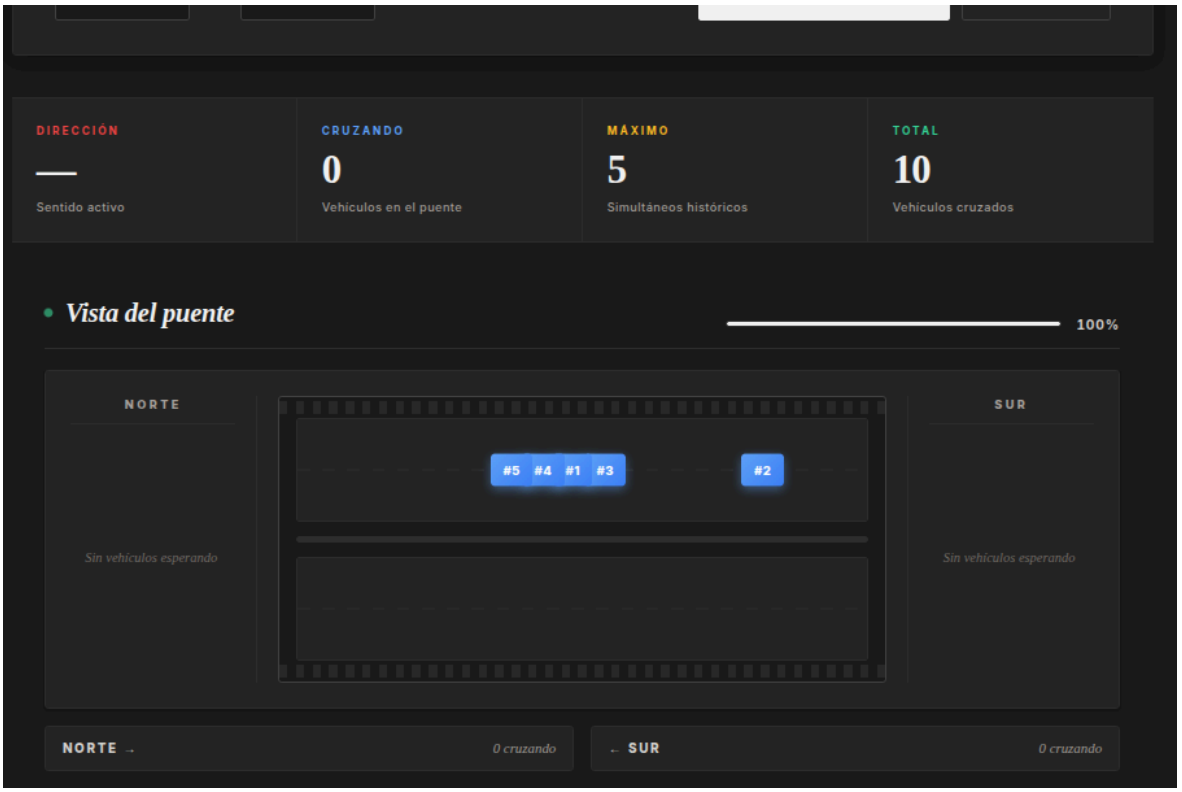
3. Organización y Estructura de Archivos

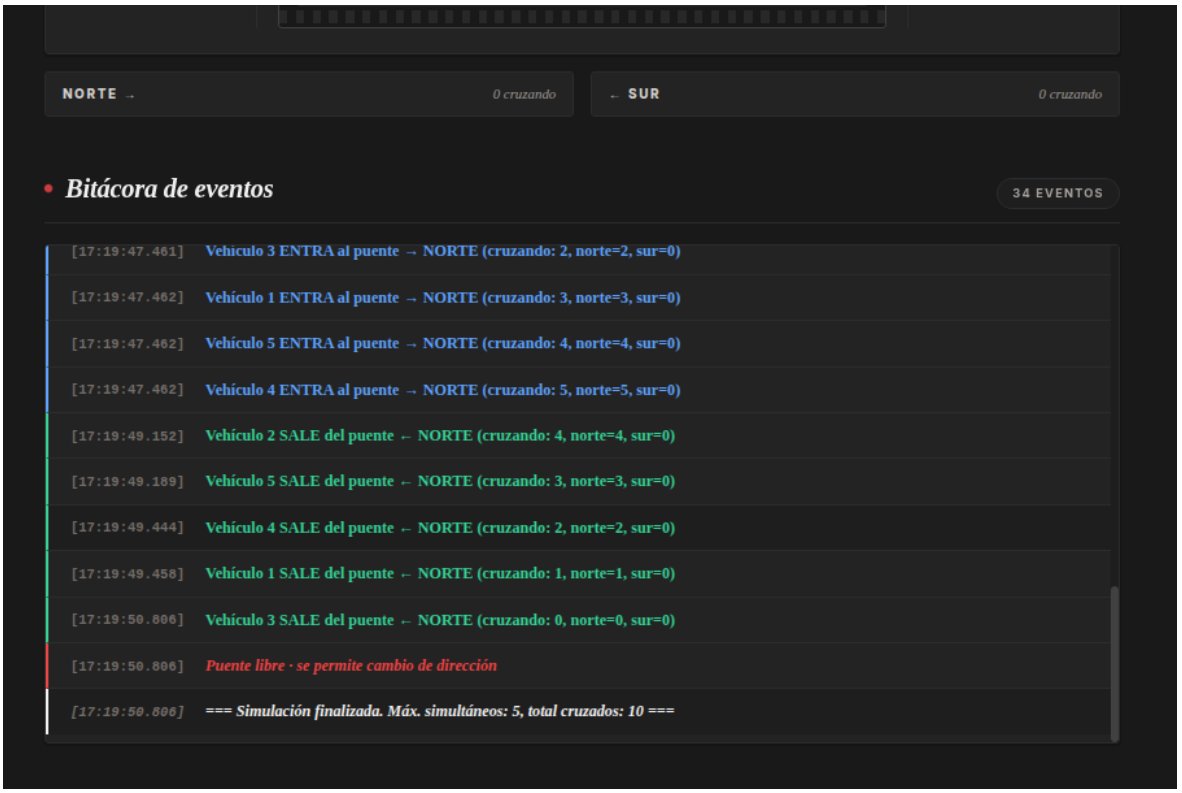


4. Interfaz


§
PUENTE CONCURRENTE
Dirección única · Prioridad compartida · Sincronización con mutex
Inactivo
CONFIGURACIÓN DEL EXPERIMENTO
VEHÍCULOS NORTE
5
VEHÍCULOS SUR
5
▶ INICIAR SIMULACIÓN
✖ LIMPIAR
DIRECCIÓN
—
Sentido activo
CRUZANDO
0
Vehículos en el puente
MÁXIMO
0
Simultáneos históricos
TOTAL
0
Vehículos cruzados
Vista del puente
0%
NORTE
SUR


DIRECCIÓN
SUR
Sentido activo
CRUZANDO
3
Vehículos en el puente
MÁXIMO
5
Simultáneos históricos
TOTAL
2
Vehículos cruzados
Vista del puente
20%
NORTE
#1
#2
#3
#4
#5
#6
SUR
Sin vehículos esperando
NORTE →
0 cruzando
← SUR
3 cruzando





Reto: modificar el algoritmo para evitar que una dirección monopolice el puente.

Para poder cumplir con el reto al agregar contadores de vehiculos que estan en espera y en la condición de entrada se añadió la regla para alternar en el caso de que la dirección actual no tenga vehículos cruzando y la otra dirección si tiene se bloquea la entrada de vehículos en la dirección actual y se cede el turno a la otra dirección lo que ayuda que no se monopolice la dirección

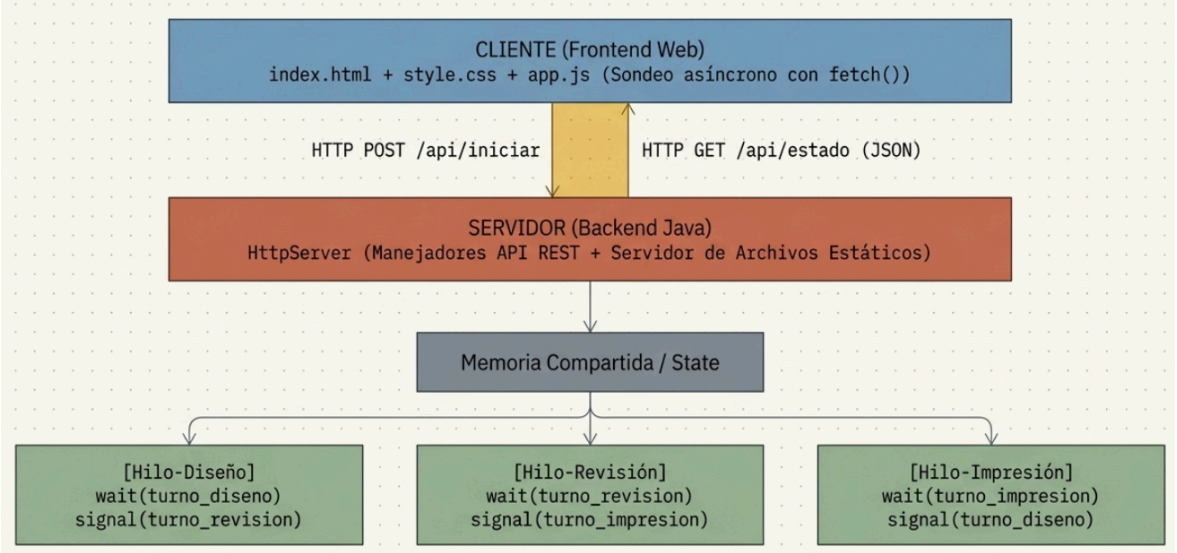
Ejercicio 5 : Centro de impresión — Sincronización de procesos

Ejercico 5

1. **Tecnologías usadas**

Este proyecto utiliza Java Standard Edition (JDK 11+) en el backend, gestionando la ejecución en paralelo de los procesos de Diseño, Revisión e Impresión mediante hilos con java.lang.Thread y garantizando la sincronización estricta de tareas con semáforos java.util.concurrent.Semaphore

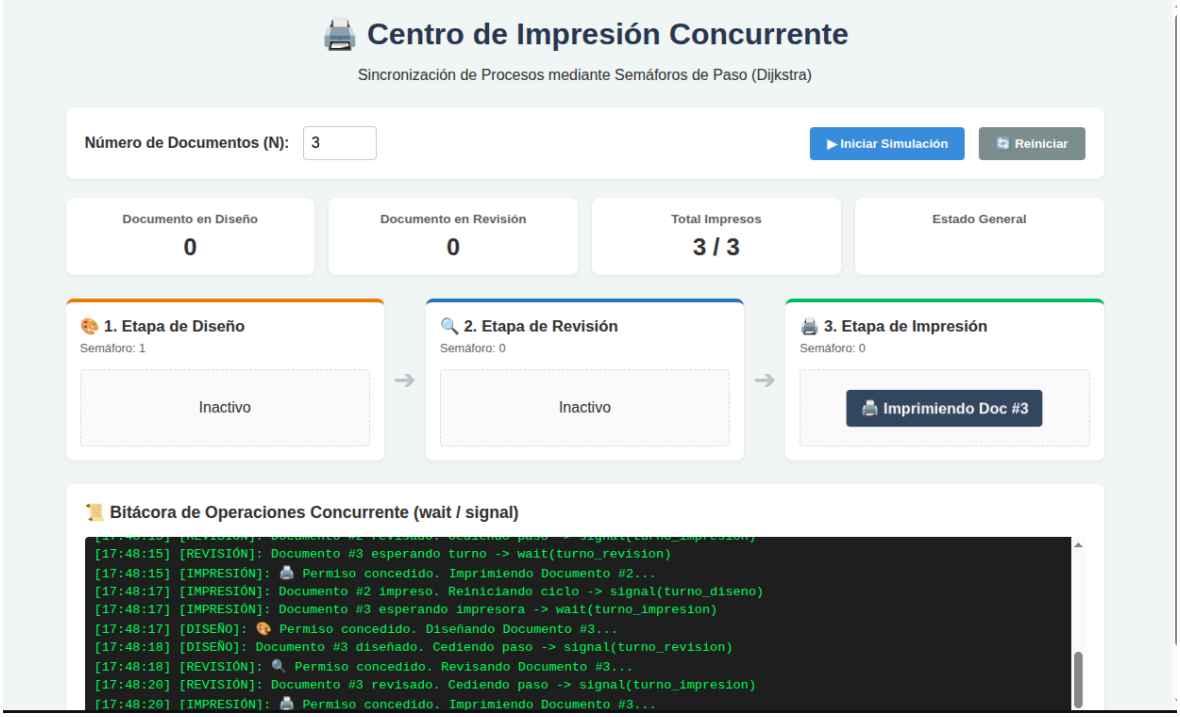
2. Arquitectura



3. Patrón de diseño usado

El sistema implementa una combinación de patrones de diseño clásicos donde la arquitectura MVC (Modelo-Vista-Controlador) estructura la aplicación al separar la lógica del pipeline concurrente y el estado compartido en memoria (ProcesoDiseno, ProcesoRevision, ProcesoImpresion)

4. Interfaz



La interfaz está dividida en cuatro bloques funcionales:

- Panel de control: donde tenemos al campo de entradas donde se puede introducir la cantidad de los documentos a procesar y con botones para iniciar o reiniciar la simulación

- Tarjeta métricas: muestran en tiempo real en que fase del procesos están los documentos
- Bitácora de logs: lleva el registro cronológico donde se visualiza todas las operaciones de sincronización

- **Comprobación de las distintas formas de cómo los hilos se comunican.**

  Se implementaron varias formas, entre ellas la comunicación mediante una cola compartida, la sincronización mediante semáforos, la exclusión mutua mediante un mutex y la comunicación entre frontend y backend mediante endpoints HTTP.
- Se espera que el estudiante mida y compare el rendimiento entre las soluciones secuenciales y concurrentes
- El estudiante debe demostrar comprensión de los conceptos vistos en cada ejercicio de la práctica

7. Preguntas de Control:

- **¿Qué es una condición de carrera?**

  Una condición de carrera es un evento que ocurre cuando dos procesos desean acceder o modificar un mismo recurso compartido en el mismo periodo de tiempo. Esta situación puede provocar un comportamiento impredecible, datos corruptos o errores en el sistema, debido a que dependerá de quien termine de ejecutarse primero en esa carrera.
- **Cuál es la diferencia fundamental entre un semáforo binario y un mutex? ¿Pueden usarse indistintamente? Justifique su respuesta con al menos dos diferencias prácticas**

  Si bien pueden parecer conceptos similares, su diferencia radica en el concepto de propiedad. Un mutex tiene un propietario estricto por lo que solo el proceso o hilo que lo adquirió puede liberarlo. Por otra parte, un semáforo binario no pertenece a ningún hilo en específico y cualquiera puede modificar su estado para señalización.

  Además, existen otras diferencias. Por ejemplo, los mutex modernos usan protocolos basados en prioridad con los que se evita que un hilo de baja prioridad bloquee indefinidamente uno de alta prioridad, algo que los semáforos no poseen.
- **Explique qué es un interbloqueo (deadlock) y enuncie las cuatro condiciones necesarias de Coffman. ¿Cuál de estas condiciones se elimina al usar la estrategia de ordenamiento de recursos?**

  Esto ocurre cuando dos o más procesos o hilos se bloquean de forma permanente porque están esperando un recurso que posee el otro, lo impide que los dos continuen con su ejecución.

  Las condiciones de coffman se deben cumplir simultáneamente y son las siguientes:
  - Exclusión mutua: Al menos un recurso debe estar retenido en un nodo.
  - Retención y espera: Un proceso que ya retiene al menos un recurso puede solicitar recursos adicionales que están siendo retenidos por otros procesos.
  - Sin expropiación: Los recursos no pueden ser usados sin que su actual proceso los libere voluntariamente.
  - Espera circular: Existe una cadena circular de dos o màs procesos en donde cada proceso espera un recurso que

tiene el siguiente proceso de la cadena, lo que genera una dependencia en cola.

Al usar la estrategia de ordenamiento se elimina la ocndiciòn de espera circular, ya que al asignar un orden numérico literal estricto a todos los recursos y obligar a los hilos a solicitar los recursos únicamente en orden ascendente, no existe la posibilidad de que se forme un ciclo cerrado de dependencias.

8. Declaración de IA

Se usó Inteligencia Artificial Generativa para realizar codificación. Esto con el modelo deepseek y siguiendo un SDD con la metodología spek-kit. Por ello existía un archivo llamado tasks.md en donde se especificaba el orden a tareas a realizar. El prompt utilizado fue:
“Lee el archivo tasks.md y comienza a realizar cada fase en relación al orden establecido”

Prompt utilizado en Gemini
Usado para entender la lógica de los hilos y como se pueden aplicar
[CONTEXTO] Actualmente me encuentro profundizando en la programación concurrente y de alto rendimiento. Necesito comprender con máxima claridad teórica y práctica cómo se gestionan los hilos (threads) de ejecución, cómo se estructuran las arquitecturas basadas en pipelines de procesamiento de datos y qué impacto directo tienen los mecanismos de sincronización en el rendimiento, la latencia y la coherencia de datos. Para aterrizar estos conceptos, requiero que todas las explicaciones y ejemplos de código se centren de manera exclusiva en el lenguaje Java. [ROL] Asume el rol de un Arquitecto Principal de Software e Ingeniero de Concurrencia en Java, con más de 20 años de experiencia diseñando sistemas distribuidos, motores de procesamiento de eventos en tiempo real y aplicaciones multihilo de ultra baja latencia. Eres reconocido por tu capacidad para explicar conceptos complejos de concurrencia de forma intuitiva sin perder el rigor técnico y por escribir código Java idiomático, robusto y listo para producción. [ACCIÓN] Desarrolla una guía técnica completa siguiendo ordenadamente estos pasos secuenciales: 1. **Fundamentos e Interacción:** - Explica el concepto de hilos (threads), el modelo de memoria de Java (JMM) y cómo se diferencia la ejecución paralela de la concurrente. - Detalla qué es un pipeline de procesamiento (modelo Productor-Consumidor / Staged Event-Driven Architecture) y cómo se conectan las etapas mediante buffers o colas de trabajo. 2. **Mecanismos de Sincronización:** - Analiza los diferentes mecanismos de sincronización en Java: `synchronized`, `ReentrantLock`, `ReadWriteLock`, variables `volatile`, operaciones atómicas (`AtomicInteger`, etc.) y barreras de sincronización (`CountDownLatch`, `CyclicBarrier`, `Phaser`).

- Explica el impacto de estos mecanismos en el rendimiento (overhead por context switching, contention, false sharing, bloqueos vs. algoritmos non-blocking/lock-free).

3. **Demostración Práctica en Java (Caso de Uso Real):**
- Construye un ejemplo completo de código Java que implemente un Pipeline multietapa (ejemplo: Lectura/Generación -> Transformación -> Persistencia/Procesamiento).
- Implementa un mecanismo de sincronización thread-safe explícito utilizando clases del paquete `java.util.concurrent` (como `ArrayBlockingQueue`, `ExecutorService` o `CompletableFuture`).
- Comenta el código detalladamente explicando dónde y por qué se aplica cada mecanismo de sincronización.

4. **Comparativa Avanzada:**
- Presenta una tabla comparativa sobre cuándo usar cada mecanismo de sincronización según el caso de uso (baja contención, alta lectura/poca escritura, paso de mensajes, etc.).
- Añade una breve explicación sobre la evolución reciente de la concurrencia en Java (Threads Tradicionales vs. Virtual Threads / Project Loom).

[FORMATO]
Estructura la respuesta utilizando formato Markdown claro y profesional:

- Usa encabezados (`##` y `###`) para separar cada sección.
- Utiliza bloques de código formatados con sintaxis de Java (` ```java `) e incluye comentarios explicativos inline.
- Emplea tablas Markdown para la comparación de mecanismos de sincronización.
- Incluye diagramas conceptuales simples en formato ASCII o texto estructurado para ilustrar el flujo de datos en el pipeline y la interacción entre hilos.

[PÚBLICO OBJETIVO]
El destinatario principal de esta respuesta es un desarrollador de software que entiende los conceptos básicos de programación pero busca dominar la concurrencia en Java a nivel profesional, comprendiendo no solo el "cómo", sino el "por qué" interno del funcionamiento del sistema.

9. Conclusiones

- A través de los ejercicios se comprueba la eficiencia de los semáforos contadores en entornos que tiene mucho de tráfico de información como se vió en el ejemplo del estacionamiento, evitando que existan la condición de carrera, también que si estos semáforos se inicializan mal la lógica del sistema se rompe llegando a ocasionar fallos o hasta la caída misma
- Con el ejemplo del cajero automático se pudo comprobar la importancia de la exclusión mutua (mutex) en sistemas donde los fallos pueden ocasionar pérdidas significativas como lo es en un banco
- El diseñar correctamente las condiciones iniciales de los semáforos es esencial para que no ocurran estados de bloqueo permanentes, muertes por inanición o condiciones de carrera como se pudo ver en el ejemplo de la línea de ensamblaje donde se debía coordinar correctamente la producción y el consumo a través de una cola limitada
- Con el ejercicio de un centro de impresión se pudo constatar que no sólo diseñar los semáforos es importante sino también los procesos de

sincronización para imponer un orden de ejecución de pipelines asegurando la distribución justa de recursos

10. Bibliografía

Semana 3. Guaman, Jose. 2026